

designers to rapidly converge upon the best implementation for power, performance and area.

Dynamic memory allocation, pointer ambiguity, memory parallelism extraction and efficient schedule creation for arbitrary C programs are all challenges for HLS. In fact, most commercial tools limit the use of pointers and dynamic memory allocation, requiring additional hardware-aware design development to create a working ASIC from a C program.

In contrast, a bespoke processor tool flow automatically creates a bespoke processor from the original, already verified gate-level netlist and application binary without further design work. Also, unlike HLS, it can generate a design that supports multiple applications on the same hardware (including full-fledged OS) and can support in-field updates. Thus a bespoke processor design flow can decrease design and verification effort and allow increased programmability compared with HLS tool flows.

Applications

Statically-specialised cores such as conservation cores, QsCores and GreenDroid automatically develop hardware implementations that are connected to a general-purpose processor at the data cache and target hotspots within an application code. GreenDroid, a prototype mobile application processor chip leverages 'dark silicon' to dramatically reduce energy consumption in smartphones. It provides many specialised processors targeting key portions of Google's Android Smartphone platform. GreenDroid reduces energy consumption for these codes by integrating conservation cores (c-cores). Such cores increase energy efficiency at the expense of increasing the total area of a design, and thus may not be a good fit for area-constrained applications.

Reconfigurable architectures such as DySER can also increase energy efficiency by mapping frequently-executed code segments onto tightly-coupled reconfigurable execution units. However, increased energy efficiency comes with an increase in area and power for the additional reconfigurable units.

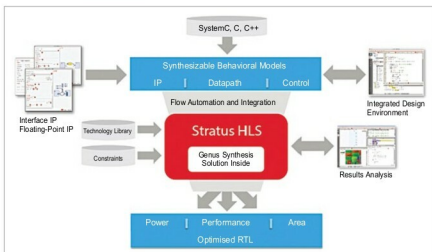


Fig. 8: Cadence Stratus high-level synthesis

Extensible processors, such as Xtensa, allow designers to specify configurations including structure sizing, optional modules and custom application-specific functional units. Such extensible processors are limited in the extent to which these can reduce area and power, since they are applied primarily at the module level. Furthermore, the process is not fully automated and requires additional hardware design effort.

Compared with extensible application-specific processors, bespoke processors can reduce power further as these can remove gates within modules and require less manual design effort. Chip generators can be used to generate chip families from the ground up for a particular application domain by allowing domain expert hardware designers to encode domain-specific knowledge into tools that design application-specific chips within the same domain.

Like HLS, this approach still requires a domain expert to design the overarching hardware in an HLS-like manner and then specify functions that allow arbitrary elaboration of the hardware design (for example, encoding optimisation functions for determining lower-level parameters such as cache associativity). Therefore chip generators require a change in the design process, while tailoring bespoke processors to applications can be completely automated from a program binary and processor netlist.

Simulate-and-eliminate attempts

to create a design tailored to an application by simulating the target application with a user-provided set of inputs on multiple base designs. Logic and interconnect components that are not used by the application are removed.

Simulate-and-eliminate differs from bespoke processors in three fundamental ways—level of automation, scope of elimination and correctness guarantees.

First, simulate-and-eliminate requires significant user input to guide the selection of core parameters, selection of bit widths and definition of optimisations. Bespoke processors require no user intervention.

Second, simulate-and-eliminate only considers high-level, manually-identified components when determining what is used by a processor, and consequently does not achieve as large of area and power reductions as fine-grained bespoke processor tailoring.

Third, simulate-and-eliminate relies on user-specified inputs to determine components that are never used by an application. This means it cannot guarantee safe optimisation for applications where inputs affect control flow. Additionally, simulate-and-eliminate cannot determine whether an unsafe elimination is performed. Bespoke processor tailoring guarantees correctness by considering all possible application inputs when determining which gates to remove.

Concluded